HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

THỰC TẬP CƠ SỞ

Weekly Report

Đề tài: XÂY DỰNG HỆ THỐNG GỢI Ý SẢN PHẨM

Giai đoạn 3: xây dựng mô hình

Giảng viên	**: Kim Ngọc Bách**
Họ và tên sinh viên	**: Nguyễn Văn Trường**
Mã sinh viên	**: B23DCCN870**
Lớp	**: D23CQCN02-B**
Nhóm	**: 11**

Hà Nội – 2026

Nội dung

I. Chia tập train/set 3
1. Đặt vấn đề 3
2. Chiến lược: Per-User Temporal Split (Chia theo thời gian của từng người dùng) 3
3. Ưu điểm 3
4. Nhược điểm 4
5. Chia tập train set 4
II. Xây dựng MF với SGD 4

I. Chia tập train/set

1. Đặt vấn đề

Trong các bài toán học máy thông thường (như phân loại ảnh, dự báo giá nhà), dữ liệu thường được chia ngẫu nhiên (Random Split) theo tỷ lệ 80/20. Tuy nhiên, áp dụng cách này vào hệ thống gợi ý phim (Recommender Systems) sẽ dẫn đến các hệ lụy nghiêm trọng:

- Rò rỉ dữ liệu (Data Leakage): Mô hình sử dụng dữ liệu đánh giá phim trong tương lai để dự đoán sở thích của người dùng trong quá khứ.
- Đánh giá ảo (Overfitting Evaluation): Kết quả kiểm thử trên tập Test đạt độ chính xác rất cao, nhưng khi triển khai thực tế lại hoạt động kém hiệu quả.

2. Chiến lược: Per-User Temporal Split (Chia theo thời gian của từng người dùng)

Khắc phục: áp dụng chiến lược chia tách dữ liệu theo trục thời gian của từng cá nhân.

- Tỷ lệ phân chia: 80% (Huấn luyện - Train) và 20% (Kiểm thử - Test).
- Nguyên tắc thực hiện: Với mỗi người dùng (User), toàn bộ lịch sử tương tác được sắp xếp theo nhãn thời gian (Timestamp). Hệ thống sẽ trích xuất 20\% tương tác cuối cùng của họ đưa vào tập Test, 80\% tương tác đầu tiên đưa vào tập Train.

  Để đảm bảo tính toàn vẹn, thuật toán áp dụng quy tắc max(1, int(len(grp) * 0.2)). Điều này đảm bảo những người dùng có ít tương tác vẫn luôn có ít nhất 1 dữ liệu ở tập Test để đánh giá.

3. Ưu điểm

- Tránh rò rỉ dữ liệu:

  Nếu chia ngẫu nhiên (Random Split), mô hình có thể vô tình học từ các hành vi trong tương lai để dự đoán quá khứ.

- Phản ánh đúng hành vi thực tế :

  Hệ thống gợi ý trong thực tế luôn vận hành theo thời gian thực. Việc lấy 20% dữ liệu cuối cùng giúp đo lường chính xác khả năng của mô hình trong việc bắt kịp sự thay đổi về sở thích của người dùng.

- Đảm bảo tính cá nhân hóa

Khác với việc chia theo một mốc thời gian cố định cho toàn bộ hệ thống (Global Split), phương pháp chia theo từng User đảm bảo:

- Mọi người dùng trong tập Test đều đã xuất hiện trong tập Train.

4. Nhược điểm

Item Cold Start: Những bộ phim mới ra mắt chỉ xuất hiện ở giai đoạn cuối (tập Test) sẽ khó được gợi ý chính xác nếu chỉ dùng thuật toán Collaborative Filtering thuần túy

⇨ Giải pháp: mở rộng mô hình, kết hợp thuật toán lai (Hybrid) có sử dụng Content-based (thể loại phim, đạo diễn) để xử lý.

5. Chia tập train set

```
TEST_RATIO = 0.20   # 80% train – 20% test

df_sorted = df_ratings.sort_values(["userId", "timestamp"])

train_list, test_list = [], []
for uid, grp in df_sorted.groupby("userId"):
    n_test = max(1, int(len(grp) * TEST_RATIO))
    train_list.append(grp.iloc[:-n_test])
    test_list.append(grp.iloc[-n_test:])

df_train = pd.concat(train_list).reset_index(drop=True)
df_test  = pd.concat(test_list).reset_index(drop=True)

print(f"\n  Tập TRAIN : {len(df_train):>7,} ratings  ({len(df_train)/len(df_ratings)*100:.1f}%)")
print(f"  Tập TEST  : {len(df_test):>7,} ratings  ({len(df_test)/len(df_ratings)*100:.1f}%)")
```

```
  Tập TRAIN :  80,367 ratings  (80.4%)
  Tập TEST  :  19,633 ratings  (19.6%)
```

II. Xây dựng MF với SGD

1. Xây dựng ma trận user-item từ tập train

```python
user_item_train = df_train.pivot_table(
    index="userId", columns="movieId", values="rating"
)

n_users  = user_item_train.shape[0]
n_movies = user_item_train.shape[1]
density  = df_train.groupby(["userId","movieId"]).ngroups / (n_users * n_movies)
sparsity = 1 - density

print(f"  Ma trận train : {n_users} users × {n_movies} movies")
print(f"  Sparsity      : {sparsity*100:.2f}%")

R_train = user_item_train.fillna(0).values.astype(np.float32)  # shape (943, 1682)
user_ids   = user_item_train.index.tolist()
movie_ids  = user_item_train.columns.tolist()
uid2idx    = {uid: i for i, uid in enumerate(user_ids)}
mid2idx    = {mid: j for j, mid in enumerate(movie_ids)}
```

2. Xây dựng MF

Ý tưởng:

Phân rã ma trận $R \approx P \times Q^T$

- P: ma trận user latent factors (n_users × K)
- Q: ma trận item latent factors (n_items × K)
- K: số latent factors (chiều ẩn)

Hàm mất mát (có regularization):

$L = \Sigma_{(u,i) \text{ rated}} (r_{ui} - p_u \cdot q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2)$

Cập nhật Stochastic Gradient Descent (mỗi rating một lần):

$e_{ui} = r_{ui} - p_u \cdot q_i$ ← residual

$p_u \leftarrow p_u + \alpha \cdot (e_{ui} \cdot q_i - \lambda \cdot p_u)$

$q_i \leftarrow q_i + \alpha \cdot (e_{ui} \cdot p_u - \lambda \cdot q_i)$

Mô hình mở rộng với bias (Simon Funk SVD):

$\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$

Cập nhật thêm: $b_u \leftarrow b_u + \alpha \cdot (e_{ui} - \lambda \cdot b_u)$

$b_i \leftarrow b_i + \alpha \cdot (e_{ui} - \lambda \cdot b_i)$

Funk SVD với bias user/item: Huấn luyện bằng Stochastic Gradient Descent.

Tham số:

K : số latent factors

alpha : learning rate

lambda_: hệ số regularization

n_epochs: số epoch

```python
class MatrixFactorization:


    def __init__(self, K=20, alpha=0.005, lambda_=0.02, n_epochs=30, seed=42):
        self.K        = K
        self.alpha    = alpha
        self.lambda_  = lambda_
        self.n_epochs = n_epochs
        self.seed     = seed



    def fit(self, df_train_data, n_users, n_items, uid2idx, mid2idx,
            df_val=None, verbose=True):
        """
        Huấn luyện mô hình trên df_train_data.
        df_val (tuỳ chọn): tính validation loss sau mỗi epoch.
        """
        np.random.seed(self.seed)
        self.uid2idx_ = uid2idx
        self.mid2idx_ = mid2idx

        # Khởi tạo ngẫu nhiên (0.1 scale)
        self.P  = np.random.normal(0, 0.1, (n_users, self.K)).astype(np.float32)
        self.Q  = np.random.normal(0, 0.1, (n_items, self.K)).astype(np.float32)
        self.bu = np.zeros(n_users, dtype=np.float32)   # user bias
        self.bi = np.zeros(n_items, dtype=np.float32)   # item bias
        self.mu = df_train_data["rating"].mean()         # global mean

        # Chuẩn bị dữ liệu train dạng array
        rows = []
        for _, r in df_train_data.iterrows():
            uid, mid, rat = int(r["userId"]), int(r["movieId"]), r["rating"]
            if uid in uid2idx and mid in mid2idx:
                rows.append((uid2idx[uid], mid2idx[mid], rat))
        rows = np.array(rows, dtype=np.float32)

        self.train_loss_history = []
        self.val_loss_history   = []

        if verbose:
            print(f"  Bắt đầu huấn luyện: K={self.K}, α={self.alpha},"
                  f" λ={self.lambda_}, epochs={self.n_epochs}")
            print(f"  {'Epoch':>6} {'Train RMSE':>12} {'Val RMSE':>10}")
```

```python
for epoch in range(1, self.n_epochs + 1):
    # Shuffle dữ liệu mỗi epoch
    np.random.shuffle(rows)
    sq_err = 0.0

    for u_idx, i_idx, r_ui in rows:
        u_idx = int(u_idx); i_idx = int(i_idx)

        # Dự đoán và residual
        r_hat = (self.mu
                 + self.bu[u_idx]
                 + self.bi[i_idx]
                 + self.P[u_idx] @ self.Q[i_idx])
        e = r_ui - r_hat
        sq_err += e * e

        # Cập nhật gradient
        p_old = self.P[u_idx].copy()
        self.P[u_idx]  += self.alpha * (e * self.Q[i_idx] - self.lambda_ * self.P[u_idx])
        self.Q[i_idx]  += self.alpha * (e * p_old           - self.lambda_ * self.Q[i_idx])
        self.bu[u_idx] += self.alpha * (e                   - self.lambda_ * self.bu[u_idx])
        self.bi[i_idx] += self.alpha * (e                   - self.lambda_ * self.bi[i_idx])

    train_rmse = np.sqrt(sq_err / len(rows))
    self.train_loss_history.append(train_rmse)

    # Validation loss
    if df_val is not None:
        val_rmse = self._eval_rmse(df_val, uid2idx, mid2idx)
    else:
        val_rmse = float("nan")
    self.val_loss_history.append(val_rmse)

    if verbose and (epoch % 5 == 0 or epoch == 1):
        print(f"  {epoch:>6}  {train_rmse:>12.4f}  {val_rmse:>10.4f}")

if verbose:
    print(f"\n  Huấn luyện xong!  Train RMSE cuối = {train_rmse:.4f}")
```

```python
def predict(self, user_id, movie_id):
    """Dự đoán rating của một cặp (user_id, movie_id)."""
    if user_id not in self.uid2idx_ or movie_id not in self.mid2idx_:
        return self.mu
    u = self.uid2idx_[user_id]
    i = self.mid2idx_[movie_id]
    pred = self.mu + self.bu[u] + self.bi[i] + self.P[u] @ self.Q[i]
    return float(np.clip(pred, 1.0, 5.0))

def _eval_rmse(self, df_val, uid2idx, mid2idx):
    """Tính RMSE trên tập validation."""
    errs = []
    for _, row in df_val.iterrows():
        uid, mid = int(row["userId"]), int(row["movieId"])
        if uid in uid2idx and mid in mid2idx:
            pred = self.predict(uid, mid)
            errs.append((row["rating"] - pred) ** 2)
    return np.sqrt(np.mean(errs)) if errs else float("nan")

def recommend(self, user_id, df_train_data, movie_title_dict, N=10):
    """Gợi ý N phim chưa xem cho user_id."""
    if user_id not in self.uid2idx_:
        return []
    seen = set(df_train_data[df_train_data["userId"] == user_id]["movieId"])
    scores = []
    for mid in self.mid2idx_:
        if mid not in seen:
            scores.append((mid, self.predict(user_id, mid)))
    scores.sort(key=lambda x: x[1], reverse=True)
    return scores[:N]
```